

Documentation of AI Interactions

Extension of Algorithmic Collusion in Large Language Models (Fish et al., 2024)

Frederik von Waldow

August 12, 2025

Abstract

During the development of the replication and extension of specific results from the study conducted by Fish et al. (2024), AI-assisted coding tools were utilized. In particular, GitHub Copilot proved valuable for code completion, error detection, and debugging. This documentation records the relevant AI interactions. A GitHub Education license was employed.

Please give me a function that calculates the monopoly price called "monopol_price". For reference consider this code snippet: ...

Here's how you can write a function to calculate the monopoly price—that is, the profit-maximizing price if a single firm (with the same demand curve) were to serve the entire market alone.

We'll follow your model and code style:

Function: `monopol_price`

```
def monopol_price(price_min=1, price_max=None, price_step=0.01, verbose=True):
    """
    Finds the monopoly price that maximizes profit for a single firm,
    using the same logit demand as in the duopoly.
    Returns:
        monopolist_price (float): Monopoly profit-maximizing price
    """
    global alpha, mu, a0, a, beta, mc, ceiling
    if price_max is None:
        price_max = ceiling
    price_grid = np.arange(price_min, price_max, price_step)
    best_profit = -np.inf
    monopolist_price = None
    for p in price_grid:
        # Both firms set same price p (as in monopoly: no competition)
        q, _ = market_demand(p, 9999, alpha) # set "other" price extremely high so demand goes
        ↪ to the monopolist
        prof = profit(p, q, alpha, mc)
        if prof > best_profit:
            best_profit = prof
            monopolist_price = p
    if verbose:
        print(f"Monopoly profit-maximizing price: {monopolist_price:.2f}")
    return monopolist_price
```

How It Works:

Loops over a grid of possible prices, as before. Calls `market_demand(p, 9999, alpha)`, setting the rival price extremely high so that essentially all the demand goes to this firm, simulating a monopoly (alternatively, you can also set both prices to p if your demand function supports a pure monopoly case). Computes profit at each price. Returns the price with maximum profit.

Take this code and please round to 2 decimal numbers: ...

```
def market_demand(price_a, price_b, alpha):
    print(f"\n=== Bertrand Competition Benchmarks (Symmetric Firms) ===")
    print(
        f"Monopoly Price = {monopol_price(session=1)[0]:.2f}, Monopoly Profit = "
        ↪ {monopol_price(session=1)[1]:.2f}")
    print(f"Nash Equilibrium Price = {nash_eq_price(verbose=False)[0]:.2f}")

    print(f"\n=== Bertrand Competition Benchmarks (Asymmetric Firms) ===")
    print(
        f"Monopoly Price = {monopol_price_asym(session=1)[0]:.2f}, Monopoly Profit = "
        ↪ {monopol_price_asym(session=1)[1]:.2f}")
    print(f"Nash Equilibrium Price: Firm 1 = {nash_eq_price_asym(verbose=False)[0]:.2f}, Firm 2 = "
        ↪ {nash_eq_price_asym(verbose=False)[1]:.2f}")
```

Consider this code that plots price and profits over round: ... I want to do the same for the "profit" column in the CSV file. I want to create these plots for each `llm_model` ranging from 1 to 6. In the end, I want a plot with 12 subplots: the left column will plot all prices for each model, and the right column will plot the profits.

Your goal is to create a 12-subplot figure (6 rows $\times$ 2 columns):

- Left column: overlayed prices for sessions 1-3 for each `llm_model` (from 1 to 6)
 - Right column: overlayed profits for sessions 1-3 for each `llm_model` (from 1 to 6)
- Let's make a clear, reusable script to achieve this. Here's how you can do it:

```
import pandas as pd
import matplotlib.pyplot as plt
import os

# Customization
N_MODELS = 6
N_SESSIONS = 3
alpha = 1 # Adjust if needed
mc = 1 # Adjust if needed
fig, axes = plt.subplots(N_MODELS, 2, figsize=(15, N_MODELS * 3), sharex=True)
for model_idx in range(1, N_MODELS + 1):
    for session in range(1, N_SESSIONS + 1):
        # Construct the file path
        csv_path = f"Prompt_0/Model_{model_idx}/monopoly_session_{session}/llm_monopoly_simulation_{session}.csv"
        if not os.path.exists(csv_path):
            print(f"File not found: {csv_path}")
            continue
        # Read CSV
        df = pd.read_csv(csv_path)
        # Plot Prices (left column)
        axes[model_idx-1, 0].plot(df["round"], df["price"] / alpha, label=f"Session {session}")
        # Plot Profits (right column)
        axes[model_idx-1, 1].plot(df["round"], df["profit"] / alpha, label=f"Session {session}")
        # Add reference lines (you might want to adapt these functions/values)
        # Calculate monopolist_price for the current model/session if needed
        try:
            monopolist_price = monopoly_price_nondifferentiated(model_idx)[0] / alpha
        except Exception:
            monopolist_price = None
        if monopolist_price is not None:
            axes[model_idx-1, 0].axhline(y=monopolist_price, color='green', linestyle=':', label='Monopoly Price')
        axes[model_idx-1, 0].axhline(y=mc, color='blue', linestyle='--', label='Marginal Cost')
        # Titles, legends
        axes[model_idx-1, 0].set_title(f"Model {model_idx}: Prices")
        axes[model_idx-1, 1].set_title(f"Model {model_idx}: Profits")
        for ax in axes[model_idx-1]:
            ax.legend()
            ax.grid(True)
            ax.set_xlabel("Round")
        axes[model_idx-1, 0].set_ylabel("Price")
        axes[model_idx-1, 1].set_ylabel("Profit")
plt.tight_layout()
plt.show()
```

I want to extract a chosen price set by a price setting LLM-agent. Please provide me with suitable code. Consider this paper for reference: "Fish, S., Gonczarowski, Y. A., & Shorrer, R. I. (2024). Algorithmic collusion by large language models. arXiv preprint arXiv:2404.00806, 7."

The paper "Algorithmic Collusion by Large Language Models" by Fish, Gonczarowski, and Shorrer (2024) presents a meticulous experimental study exploring how Large Language Models (LLMs), specifically GPT-4, can autonomously engage in collusive pricing behaviors in simulated market environments.

Retrieving Agent Choices from LLMs

Process:

1) Prompting: Each agent is provided with a structured prompt.

2) LLM Response: The LLM generates a textual response containing:

Reasoning: A narrative explaining the pricing decision.

Price Output: The proposed price for the current period.

3) Extraction: The numerical price is extracted from the LLM's textual output using regular expressions.

Example Extraction Code:

```
import re
def extract_price(llm_output):
    match = re.search(r'price(?: is|:)?\s*\$(\d+(\?:\.\d{1,2})?)', llm_output, re.IGNORECASE)
    if match:
        return float(match.group(1))
    else:
        raise ValueError("Price not found in LLM output.")
```

Note: This function searches for patterns like "price is 10.50" or "price: \$10.50" and extracts the numerical value.

Consider this code: ... Please store the output in a CSV file with detailed data.

Below is how to store the simulation output to a CSV file that includes full details for each round, including:

- Round number
- Each agent's price, quantity sold, profit
- The plan text each agent gave for their strategy

Place this code after the simulation loop, or at the end of your script:

```
import csv
# File name for output
output_file = "llm_duopoly_simulation.csv"
# Combine full row data including plans
with open(output_file, mode='w', newline='', encoding='utf-8') as file:
    writer = csv.writer(file)
    # Write header
    writer.writerow([
        "round",
        "agent_0_price", "agent_1_price",
        "agent_0_quantity", "agent_1_quantity",
        "agent_0_profit", "agent_1_profit",
        "agent_0_plan", "agent_1_plan"
    ])
    # Write row for each round
    for i, row in enumerate(history):
        writer.writerow([
            row['round'],
            row['price_0'], row['price_1'],
            row['qty_0'], row['qty_1'],
            row['profit_0'], row['profit_1'],
            plans[0], plans[1] # latest plans from last round
        ])
    )
```

Note that I have $a=2$ and $a_2=2.75$. I want that $a=2$ corresponds to $i=0$ and $a_2=2.75$ to $i=1$. Please adjust this the given demand function accordingly: ...

Here's how you can adjust the function so that for $i=0$ it uses $a=2$ and for $i=1$ it uses $a_2=2.75$:

```
def market_demand(price_a, price_b, alpha):
    a = 2
    a2 = 2.75
    prices = [price_a, price_b]
    a_list = [a, a2]
    denom = sum(np.exp((a_list[i] - prices[i] / alpha) / mu) for i in range(2)) + np.exp(a0 /
    ↪ mu)
    q = [beta * np.exp((a_list[i] - prices[i] / alpha) / mu) / denom for i in range(2)]
    return q[0], q[1]
```

Why does this code ... returns Nash prices which are identical across firms? In this case the firms are asymmetric, that is $a=2$, $a_2=2.75$.

Your code provides the same Nash prices for both firms because of how you call `market_demand` inside the loop:

```
q1, _ = market_demand(p1, p2, alpha)
q2, _ = market_demand(p2, p1, alpha) # If asymmetric
```

What's happening: `q1` is always the demand for firm 1 (with $a=2$), and `q2` is always the demand for firm 1 again (with $a=2$), just with prices swapped. How to fix: Call `market_demand(p1, p2, alpha)` once per price pair, and use its two outputs for the two firms:

```
for i, p1 in enumerate(prices):
    for j, p2 in enumerate(prices):
        q1, q2 = market_demand(p1, p2, alpha) # Only call once!
        prof1 = profit(p1, q1, alpha, mc)
        prof2 = profit(p2, q2, alpha, mc)
        matrix[i, j, 0] = prof1
        matrix[i, j, 1] = prof2
```

This way, the asymmetry in `a` and `a2` will be respected, and the Nash prices will generally differ.

I want `history_strings` to be ordered from latest to oldest entry. Then, I want to access the 100 latest entries. For example, in round 141 I want to access rounds 41 to 140 and not from 1 to 100.

Given your requirements:

You want the list ordered latest to oldest (i.e., index 0 is the newest). You want to access the 100 newest entries (i.e., indices 0 to 99). In round 141, you want to access rounds 41 to 140 (the 100 most recent, excluding the current round).

How to do this:

1. Insert new entries at the beginning (as you do):

```
history_strings[0].insert(0, f"Round {round_num}: ...")
```

2. To access the 100 newest entries (excluding the current round):

```
recent_history = "\n\n".join(history_strings[0][1:101])
```

`history_strings[0][1:101]` gives you entries at indices 1 to 100 (i.e., the 100 rounds before the current one). This will be ordered from newest to oldest. If you want them in chronological order (oldest to newest):

```
recent_history = "\n\n".join(reversed(history_strings[0][1:101]))
```

Based on this code: ..., please implement some code such that each output for a given round and `agent_id` is saved in the following format: "output_round1_agent1.txt"

To save each LLM response (`output`) from every round and `agent_id` in the format:
output_round1_agent1.txt
output_round23_agent0.txt
You can add a file-writing line inside your main loop where the agent response is handled.
Place it right after:

```
print(f"\nAgent {agent_id} response:\n{output}")
```

Add:

```
# Save output to file for later inspection
output_filename = f"output_round{round_num}_agent{agent_id}.txt"
with open(output_filename, "w", encoding="utf-8") as f:
    f.write(output)
```

Consider this code: ... Make sure the x-axis only ranges from -40 to 40.

You can set the x-axis range using `plt.xlim(-40, 40)`. Add this line before `plt.tight_layout()`:

```
plt.figure(figsize=(8,8))
plt.scatter(profit_diffs_P1, profit_sums_P1, marker='x', color='blue', label='P1 vs. P1')
plt.axhline(y=monopol_profit, color='green', linestyle='--')
plt.plot(profit_diff, sum_nash_profits, 'r--', linewidth=2)
plt.xlabel(f'Average difference in profits 1 - 2 (last {last_rounds} rounds)')
plt.ylabel(f'Average sum of profits 1 + 2 (last {last_rounds} rounds)')
plt.title('P1 vs P2: Profits Earned')
plt.text(-42, monopol_profit, r'$\pi^m$', color='green', fontsize=12, ha='center',
        ↪ va='bottom', rotation=0)
plt.text(-31, 58, r'$\pi_1 = \pi^{\text{Nash}}$', fontsize=12, color='red')
plt.text(18, 58, r'$\pi_2 = \pi^{\text{Nash}}$', fontsize=12, color='red')
plt.legend()
plt.xlim(-40, 40) # Set x-axis range
plt.tight_layout()
plt.show()
```

Please add a dashed diagonal line to this plot: ...

```
plt.figure(figsize=(8, 8))
plt.scatter(prices0_P1, prices1_P1, marker="x",
            color="blue", label="P1 vs. P1")
plt.axhline(y=nash_price, color="red", linestyle="--", label = r"$p^{\text{nash}}$")
plt.axvline(x=nash_price, color="red", linestyle="--")
plt.axhline(y=monopolist_price, color="green", linestyle="--", label = r"$p^{\text{m}}$")
plt.axvline(x=monopolist_price, color="green", linestyle="--")
# Add dashed diagonal line
min_val = min(min(prices0_P1), min(prices1_P1))
max_val = max(max(prices0_P1), max(prices1_P1))
plt.plot([min_val, max_val], [min_val, max_val], linestyle="--", color="gray",
        ↪ label="Diagonal")
plt.xlabel(f"Firm 1 average price (last {last_rounds} rounds)")
plt.ylabel(f"Firm 2 average price (last {last_rounds} rounds)")
plt.title("P1 vs P1: Pricing Behavior")
plt.legend()
plt.savefig(f"Bertrand_pricing_p1_results.pdf")
plt.show()
```

Please plot these figures side by side based in this code: ...

```
# ...existing code...
fig, axes = plt.subplots(1, 2, figsize=(16, 7))
# --- Figure 1: Pricing Behavior ---
ax = axes[0]
min_val = min(min(prices0_P1), min(prices1_P1))
max_val = max(max(prices0_P1), max(prices1_P1))
ax.plot([min_val, max_val], [min_val, max_val], linestyle="--",
        color="black", linewidth=0.5, alpha=1.0)
ax.scatter(prices0_P1, prices1_P1, marker="x",
           color="blue", label="P1 vs. P1")
ax.axhline(y=nash_price, color="red", linestyle="--", label=r"$p^{\text{nash}}$")
ax.axvline(x=nash_price, color="red", linestyle="--")
ax.axhline(y=monopolist_price, color="green",
           linestyle="--", label=r"$p^{\text{m}}$")
ax.axvline(x=monopolist_price, color="green", linestyle="--")
ax.set_xlabel(f"Firm 1 average price (last {last_rounds} rounds)")
ax.set_ylabel(f"Firm 2 average price (last {last_rounds} rounds)")
ax.set_title("P1 vs P1: Pricing Behavior")
ax.legend()
# --- Figure 2: Profit Behavior ---
ax = axes[1]
monopol_profit = monopoly_price(session=1)[1]
nash_profit_sum = 2 * profit(
    nash_price, market_demand(nash_price, nash_price, alpha)[0], alpha, mc
)
profit_diff = np.linspace(-23.12, 23.12, 500)
sum_nash_profits = nash_profit_sum + np.abs(profit_diff)

ax.axvline(x=0, color="black", linestyle="-", linewidth=0.5)
ax.scatter(
    profit_diffs_P1, profit_sums_P1, marker="x", color="blue", label="P1 vs. P1"
)
ax.axhline(y=monopol_profit, color="green",
           linestyle="--", label=r"$\pi^{\text{m}}$")
ax.plot(profit_diff, sum_nash_profits, "r--", linewidth=2)
ax.set_xlabel(f"Average difference in profits 1 - 2 (last {last_rounds} rounds)")
ax.set_ylabel(f"Avgerage sum of profits 1 + 2 (last {last_rounds} rounds)")
ax.set_title("P1 vs P1: Profits Earned")
ax.text(-20, 50, r"$\pi_1 = \pi^{\text{Nash}}$", fontsize=12, color="red")
ax.text(11, 50, r"$\pi_2 = \pi^{\text{Nash}}$", fontsize=12, color="red")
ax.legend()
ax.set_xlim(-30, 30)
plt.tight_layout()
plt.savefig("Bertrand_pricing_p1_side_by_side.pdf")
plt.show()
# ...existing code...
```

Consider this code: ... I want to check the percentage of valid plan and insight entries that do not say: "No plan found" and "No insight found" for each session.

You can print the percentage of valid outputs (i.e., not "No plan found." or "No insight found.") for each session by modifying your code as follows:

```
for llm_model in [1, 2, 3, 4, 5, 6]:
    model = model_MAP.get(llm_model)
    print(f"Model: {model}")
    for session in range(1, 4):
        print(f"\n    Session: {session}")
        df = pd.read_csv(
            f"Prompt_0/Model_{llm_model}/monopoly_session_{session}/
            ↳ llm_monopoly_simulation_{session}.csv"
        )
        # Filter session and periods 201-300
        sub = df[df["round"].between(201, 300)]
        prices = sub["price"].values
        converged, p_found = check_convergence(prices)
        print(
            f"    Converged: {converged}, Found Price: {round(p_found, 2) if p_found is not None
            ↳ else p_found}"
        )
        monopoly_converged, monopoly_p = monopoly_price_convergence(prices)
        print(
            f"    Monopoly Converged: {monopoly_converged}, Monopoly Price: {round(monopoly_p, 2)
            ↳ if monopoly_p is not None else monopoly_p}"
        )
        # Calculate valid output percentages
        total = len(sub)
        no_plan = (sub["plan"] == "No plan found.").sum()
        no_insight = (sub["insight"] == "No insight found.").sum()
        valid_plan_pct = 100 * (total - no_plan) / total if total > 0 else 0
        valid_insight_pct = 100 * (total - no_insight) / total if total > 0 else 0
        print(
            f"    Valid plan entries: {total - no_plan}/{total} = {valid_plan_pct:.2f}%"
            if total > 0 else "    No data"
        )
        print(
            f"    Valid insight entries: {total - no_insight}/{total} = {valid_insight_pct:.2f}%"
            if total > 0 else "    No data"
        )
    )
```

This will print the percentage of valid (non-missing) plan and insight outputs for each session.

Consider the dataframe `pairs`. Please remove all duplicate entries based on the combination of `round`, `session`, `asym`, alternating between agent 1 and agent 0. For example, for `session = 1` and `asym = 0`: in round 102, retain only the entry for agent 0; in round 104, retain only the entry for agent 1; in round 106, retain only the entry for agent 0; and so on.

You can achieve this by:

- 1) Sorting your DataFrame by `session`, `asym`, and `round`.
 - 2) For each group (`session`, `asym`), assign an alternating agent value: 0, 1, 0, 1, ... (starting with 0).
 - 3) Keep only the rows where the `agent` matches the assigned value for that row.
- Here's how you can do it in pandas:

```
# Drop every duplicate (round, session, asym) alternating between agent 0 and 1 row
def alternating_agent_filter(df):
    # Sort by session, asym, round, agent for consistency
    df = df.sort_values(['session', 'asym', 'round', 'agent']).reset_index(drop=True)
    # For each (session, asym), assign an alternating agent: 0,1,0,1,...
    def filter_group(group):
        # Get unique rounds in order
        rounds = group['round'].unique()
        keep_rows = []
        for idx, rnd in enumerate(rounds):
            agent_to_keep = idx % 2 # 0,1,0,1,...
            # Keep only the row with the correct agent for this round
            row = group[(group['round'] == rnd) & (group['agent'] == agent_to_keep)]
            keep_rows.append(row)
        return pd.concat(keep_rows)
    filtered = df.groupby(['session', 'asym'],
        ↪ group_keys=False).apply(filter_group).reset_index(drop=True)
    return filtered
pairs = alternating_agent_filter(pairs)
```

This will keep, for each (`session`, `asym`), only one agent per round, alternating between agent 0 and 1 as you described.

Take this code: Conduct separate regressions for agent 0 and agent 1 for each `asym` value.

Certainly! Here's how you can extend your fixed effects regression loop to also run separate regressions for agent 0 and agent 1 for each `asym` value. This will give you three regressions per `asym`: one for both agents together (as before), one for agent 0, and one for agent 1.

```
for asym in range(0, 2):
    print(f"\n--- Fixed Effects Regression for Asym = {asym} (Both Agents) ---")
    pairs_asym = pairs[pairs["asym"] == asym].copy()
    pairs_asym["entity"] = (
        pairs_asym["session"].astype(str) + "_" + pairs_asym["agent"].astype(str)
    )
    pairs_asym = pairs_asym.set_index(["entity", "round"])
    y = pairs_asym["price"]
    x = pairs_asym[["price.L1", "comp_price.L1"]]
    model = PanelOLS(y, x, entity_effects=True)
    results = model.fit()
    print(results.summary)

# Separate regressions for agent 0 and agent 1
for agent in [0, 1]:
    print(f"\n--- Fixed Effects Regression for Asym = {asym}, Agent = {agent} ---")
    pairs_agent = pairs_asym[pairs_asym["agent"] == agent].copy()
    if len(pairs_agent) < 10:
        print("Not enough data for regression.")
        continue
    y_agent = pairs_agent["price"]
    x_agent = pairs_agent[["price.L1", "comp_price.L1"]]
    model_agent = PanelOLS(y_agent, x_agent, entity_effects=True)
    results_agent = model_agent.fit()
    print(results_agent.summary)
```

How it works:

For each `asym` value, you run the original regression for all agents. Then, for each agent (0 and 1), you filter the data and run a separate regression. If there are too few rows for a regression, it will skip and print a warning.

You can paste this code in place of your current regression loop.

Explain in detail why the dataframe `pairs` contains duplicate entries.

The `pairs` DataFrame has double entries (i.e., two rows per agent per round) because of how the data is reshaped from wide to long format and how the lagged competitor price is merged.

Why double entries?

1) Long format for both agents:

In the code, after melting, each row in `long_df` represents the outcome for one agent in one round. Since there are two agents per session and round, you get two rows per session-round pair.

2) Lagged competitor price merge:

When you merge the competitor's lagged price `comp_price.L1`, you align each agent's row with their competitor's previous price. This does not remove the double entries; it just adds the competitor's info to each agent's row.

3) Filtering for even rounds:

The line `pairs = long_df[long_df["round"] % 2 == 0].copy()` keeps only even rounds, but still for both agents. So for each session and even round, you have one row for agent 0 and one for agent 1.

Example:

Suppose you have session 1, round 2:

Row 1: agent 0, round 2, price, price.L1, comp_price.L1

Row 2: agent 1, round 2, price, price.L1, comp_price.L1

This is by design: each row is an observation for one agent in one round, with their own lagged price and their competitor's lagged price.

Why does this code: ... create duplicate entries in the dataframe `pairs` ?

This code can create duplicate entries in `long_df` (and thus in `pairs`) because of the way the competitor's lagged price is merged:

```
comp_df = long_df[["session", "agent", "round", "price"]].copy()
comp_df = comp_df.rename(columns={"agent": "comp_agent", "price": "comp_price.L1"})
comp_df["round"] += 1
long_df = pd.merge(long_df, comp_df, on=["session", "comp_agent", "round"], how="left")
```

Why duplicates occur

The merge is performed on `["session", "comp_agent", "round"]`. For each row in `long_df`, if there are multiple rows in `comp_df` with the same `session`, `comp_agent`, and `round`, the merge will create a Cartesian product for those matches, resulting in duplicate rows. This can happen if, for example, there are multiple agents per session and round, or if the data is not uniquely indexed by these columns.

Example

Suppose you have two agents (0 and 1) for the same session and round. After shifting and renaming, both could match the same `comp_agent` and `round` in the merge, causing two rows to become four.

How to avoid

Ensure that for each combination of `session`, `comp_agent`, and `round`, there is only one row in `comp_df`. Or, after the merge, drop duplicates if appropriate, or check your merge keys to ensure uniqueness.

Consider this regression output ..., please provide a markdown regression table containing all coefficients.

Here is a markdown regression table summarizing all coefficients from your regression output:

Model / Spec	Variable	Coefficient	Std. Error	t / z-stat	P-value	95% CI Lower	95% CI Upper
↪ 95% CI Upper							
FE - Asym = 0 (Both Agents)	price.L1	0.0102	0.0124	0.8231	0.4105	-0.0141	0.0344
	comp_price.L1	0.0012	0.0034	0.3470	0.7287	-0.0054	0.0078
OLS - Asym = 0 (Both Agents)	const	1.6373	0.511	3.202	0.001	0.635	2.639
	price.L1	0.6547	0.110	5.959	0.000	0.439	0.870
	comp_price.L1	0.1658	0.066	2.504	0.012	0.036	0.296
FE - Asym = 1 (Both Agents)	price.L1	-0.0120	0.0304	-0.3934	0.6941	-0.0716	0.0477
	comp_price.L1	0.0041	0.0071	0.5872	0.5571	-0.0097	0.0180
OLS - Asym = 1 (Both Agents)	const	0.8736	0.267	3.270	0.001	0.350	1.397
	price.L1	0.7480	0.090	8.302	0.000	0.571	0.925
	comp_price.L1	0.1826	0.077	2.367	0.018	0.031	0.334
FE - Asym = 1, Agent = 0	price.L1	-0.0018	0.0334	-0.0548	0.9563	-0.0673	0.0636
	comp_price.L1	-0.0048	0.0074	-0.6455	0.5187	-0.0193	0.0097
OLS - Asym = 1, Agent = 0	const	1.1544	0.461	2.503	0.012	0.251	2.058
	price.L1	0.7421	0.125	5.916	0.000	0.496	0.988
	comp_price.L1	0.1720	0.101	1.697	0.090	-0.027	0.371
FE - Asym = 1, Agent = 1	price.L1	-0.0331	0.0604	-0.5487	0.5833	-0.1517	0.0854
	comp_price.L1	0.0230	0.0103	2.2353	0.0256	0.0028	0.0431
OLS - Asym = 1, Agent = 1	const	0.5854	0.257	2.281	0.023	0.082	1.088
	price.L1	0.7455	0.106	7.031	0.000	0.538	0.953
	comp_price.L1	0.2023	0.093	2.169	0.030	0.020	0.385

This table:

- Puts all models together so you can compare across specifications.
- Preserves FE vs. OLS differences.
- Keeps coefficients, SEs, t/z-stats, p-values, and CIs aligned.

If you'd like, I can also produce a LaTeX table version for academic paper formatting.

I want to get sentence embedding via `nlTK` following this code :

```
"def get_embedding(text, model="mx-bai-embed-large:latest"):  
    response = client.embeddings.create(  
        model=model,  
        input=text  
    )  
    return np.array(response.data[0].embedding)"
```

Your function is set up to get embeddings using a model (e.g., "`mx-bai-embed-large:latest`") via an API client called `client`.

Here's what you need to do to get sentence embeddings for each sentence tokenized in your previous code.

Assumptions:

- You have `client` properly set up (e.g., for OpenAI or a similar provider).
- You have imported `numpy as np`. Full Example: From Text File to Sentence Embeddings

```
import nltk  
import numpy as np  
  
# Download the punkt tokenizer if not already present  
nltk.download('punkt')  
# Your embedding function  
def get_embedding(text, model="mx-bai-embed-large:latest"):  
    response = client.embeddings.create(  
        model=model,  
        input=text  
    )  
    return np.array(response.data[0].embedding)  
# Read the file  
with open(r'C:\Users\...\output_model\output_round1.txt', 'r', encoding='utf-8') as f:  
    text = f.read()  
# Tokenize into sentences  
from nltk.tokenize import sent_tokenize  
sentences = sent_tokenize(text)  
# Get embeddings for each sentence  
embeddings = []  
for sentence in sentences:  
    emb = get_embedding(sentence)  
    embeddings.append(emb)  
# Optionally: convert to a 2D numpy array  
embeddings = np.vstack(embeddings) # shape: (num_sentences, embedding_dim)  
# Now 'embeddings' is a matrix where each row is a sentence embedding
```

Print first 3 entries considering the following code: ...

To print only the first 3 entries for each list, slice the lists in your for-loops:

```
for sentence, sim in sorted_results_avoid_p1[:3]:  
    print(f"{sentence[:80]}... similarity: {sim:.4f}")  
for sentence, sim in sorted_results_avoid_p2[:3]:  
    print(f"{sentence[:80]}... similarity: {sim:.4f}")  
for sentence, sim in sorted_results_start_p1[:3]:  
    print(f"{sentence[:80]}... similarity: {sim:.4f}")  
for sentence, sim in sorted_results_start_p2[:3]:  
    print(f"{sentence[:80]}... similarity: {sim:.4f}")
```